

吴天凯

☎ 173-4201-9213 ✉ wutiankai179@qq.com

github.com/tiankai-333 npm: @wutiankai/npc-dialogue

求职意向: AI 应用工程师 / Java 后端开发 · 2027 届校招



👤 个人简介

东华大学软件工程 2027 届，AI 应用与 Java 后端方向。主导教育 AI 学习平台从 Spring Boot 2 / Java 8 的手写 LLM、RAG 与规则编排，演进到 Java 21、Spring Boot 4、Spring AI 2 的模型、记忆、检索、工具与 MCP 架构；另有已上线 AI 文字冒险游戏。能够从业务问题出发完成 AI 能力设计、全栈实现、数据迁移、权限治理、评测观测和部署，并能用对照实验解释技术选型的收益与代价。

⚙️ 技能

- **AI 应用框架**: Spring AI (ChatClient、ChatModel、Advisor、ChatMemory、VectorStore、Tool Calling、MCP); LangChain4j (AI Services、Tool、Content Retriever); LlamaIndex 文档解析与增量入库。
- **Agent 与上下文**: 理解 LLM 无状态、上下文窗口、短期/长期记忆、ReAct、Planner/Executor、工具执行循环、结构化输出、人工审批与 Agent 权限边界。
- **RAG 与数据**: Embedding、语义分块、Top-K/阈值检索、metadata、引用溯源、降级策略; Redis VectorStore、Qdrant、MySQL; 了解稀疏/稠密检索与重排序。
- **Java 后端**: Java 21、Spring Boot 4、Spring MVC / WebFlux、MyBatis、Spring Security、MySQL、Redis、SSE、REST、Maven 多模块、Micrometer / Actuator。
- **Python 与前端**: Python、FastAPI、LlamaIndex、pdfplumber; Vue 3、Next.js、TypeScript、微信小程序; 可完成 AI 应用前后端闭环。
- **工程与部署**: Docker Compose、Nginx、Git、Linux; 熟悉 A/B 迁移、Mock/端到端测试、token/费用/延迟观测、密钥加密与公开仓库安全。
- **模型基础**: 理解 Transformer、Attention、Embedding 与编码器-解码器; 完成 BERT / LSTM 中文 NER、BART 文本生成训练与评估实验。

🔧 项目经历

408Master: 面向计算机考研 408 的 AI 学习平台

项目队长 · 校实训二等奖

Java 21 / Spring Boot 4 / Spring AI 2 Redis / Qdrant / MySQL Python / LlamaIndex / Vue 3

github.com/tiankai-333/master408

项目简介: 在开源考试系统基础上完成面向考研 408 的深度二次开发，覆盖题库、考试、错题、个人画像、知识图谱、AI 解析、AI 组卷、Web 学生/管理端与微信小程序。在产品闭环上线后继续治理模型调用、对话状态、检索和编排等架构问题，引入 Spring AI、ChatMemory、VectorStore、LangChain4j、LlamaIndex 和 MCP，形成“Java 负责在线查询与业务副作用、Python 负责离线解析入库、模型负责语义决策、确定性代码负责执行”的 AI 应用架构。

核心职责:

- 负责项目架构、技术选型和里程碑拆分; 审计约 279 个 Java 文件、41 个领域类、27 个 Mapper 与 26 个 Mapper XML，按“原样复用、机械升级、保留契约替换实现、淘汰”四档迁移，保持前端 API 和题库业务兼容。
- 设计 Java 主应用 + Python 入库边车 + MCP 运维智能体的边界; 为每个阶段建立改造前后对比文档，记录功能质量、p50/p95/p99、token 成本、错误率、改动面及下一阶段决策。
- 负责 AI 模型接入、Prompt、Memory、RAG、Tool Calling、Provider 路由、使用日志、安全与观测; 保留 Vue 3、微信小程序、爬虫和 HTML/图片题目渲染资产。

主要模块:

- **Spring AI 调用层**: 使用 ChatClient 取代手写 HttpURLConnection / RestTemplate 和 SSE 解析; 以 ChatModel 适配智谱、DeepSeek、OpenAI 兼容模型, 支持同步、流式和多模态题目识别。
- **多供应商路由**: 复用数据库中的 Provider 配置、enabled/priority、用户私有 key 和模型字段, 封装可替换 ChatModel; 公共与个人密钥均以 AES-GCM 加密, 主密钥由环境变量注入。
- **可观测调用链**: 通过 Advisor / Observation 统一采集模型、token、费用、耗时、缓存命中、成功状态和错误原因, 复用原使用日志表与管理后台统计。
- **柏拉图多轮教学**: 使用 Redis ChatMemoryRepository 持久化历史, 以 userId + sessionKey 隔离会话; 每轮只提出下一个关键问题, 通过消息窗口、摘要和主动揭示答案的 Tool 控制上下文。
- **RAG 查询层**: 以 Spring AI VectorStore 统一语义检索, 保留 metadata、引用溯源和向量库不可用时降级直答; 移除把数千条向量常驻 JVM 的 MySQL 内存余弦兜底。
- **LlamaIndex 入库边车**: FastAPI 提供 parse/ingest/reindex 接口, 负责 PDF/HTML 解析、OCR/本地解析降级、语义分块、图片下载与链接重写, 并将 chunk 元数据写入 MySQL、向量写入统一存储。
- **真实 Tool Calling**: 把错题查询、题库检索、确定性组卷和学习计划能力封装为 Tool; 模型决定是否调用及参数, Java 服务执行查询、事务和副作用, 调用日志记录真实参与结果。
- **MCP 运维智能体**: 暴露健康、脱敏配置、限量日志、指标和 AI 使用统计等只读工具; Agent 生成诊断报告, 清缓存、重建索引、重启等破坏性操作必须进入 Planner/Executor 工单并人工审批。

💡 技术难点与解决方案

难点 1: 旧系统业务资产多, 直接重写成本高, 框架升级又容易同时改变接口与行为

项目已包含完整题库、组卷、用户、前端和数据资产, 早期技术基线为 Java 8 / Spring Boot 2.1, 升级到 Boot 4 涉及 Jakarta、Spring Security、MyBatis 与依赖迁移。如果同时重写业务和 AI 层, 将无法判断问题来自框架、模型还是业务变化。

解决方案: 采用渐进式迁移与兼容层。保留 'com.mindskip.xzs' 包名、Controller URL、DTO、Mapper/XML 和核心 Service, 先机械完成 Java 21 / Jakarta 迁移; 旧模型调用与 Spring AI 新实现通过开关并存, 用相同 Prompt、模型、参数和问题集进行 A/B, 验证稳定后再删除旧传输层。模块边界按真实依赖逐步抽取, 不为追求目录整齐提前重写。

难点 2: LLM 无状态, 一次性启发式 Prompt 无法形成真正的苏格拉底式多轮教学

早期柏拉图 Prompt 同时要求“不直接给答案”和“展示完整自问自答及最终答案”, 且下一轮仅由前端拼接有限历史, 刷新后上下文丢失, 无法根据学生回答逐层调整问题。

解决方案: 重写为单步追问 Prompt, 每轮只输出一个关键问题; 通过 Redis ChatMemory 按会话持久化 user/assistant 消息, Advisor 自动加载历史; 设置消息窗口、过期时间、摘要策略与会话隔离, 学生主动要求答案或达到终止条件后再调用 RevealAnswer Tool, 明确“记忆存储”和“教学状态机”的职责。

难点 3: RAG 检索链路重复, 向量常驻 JVM, 数据写入与在线查询职责混杂

早期 RAG 同时维护 Qdrant 与 MySQL 全量向量余弦检索, 存在两套阈值、Top-K 和降级逻辑; 文档解析、向量入库和在线查询耦合, 新增 PDF 或 HTML 资料难以持续增量更新。

解决方案: 将数据真相定义为 document/chunk/metadata, Python LlamaIndex 边车只负责原始文档到 chunk 与向量的写入, Java Spring AI 只负责查询时检索; VectorStore 统一 Top-K、相似度阈值和过滤条件, 引用结果通过原 SSE 协议返回前端。保留 Qdrant 作为 LangChain4j 对照后端, 并通过固定数据集比较 Redis/Qdrant 的接入复杂度与检索行为。

难点 4: 规则编排稳定但覆盖有限, 完全交给 LLM 又会增加延迟、不确定性和副作用风险

早期正则路由对显式命令有效, 但难以识别自由表达; 组卷/规划由 SQL 服务直接执行, 并未形成真实工具调用。若直接让模型控制所有流程, 又可能出现误调用、参数错误、循环和越权操作。

解决方案: 使用混合路由: 显式 intent 和高置信规则走确定性 fast path, 自由文本交给 Tool Calling; Tool 仅暴露窄参数 schema, 限制调用次数、超时和权限, 事务操作由 Java 服务负责。模型只能提出动作, 不能直接修改数据库; 高风险工具要求人工确认, 真实 tool call 全链路留痕。

难点 5：引入框架容易只得到“代码更整齐”的主观结论，无法证明值得

模型推理和网络通常占主要延迟，Spring AI 不必然更快；Memory、RAG 与工具循环还会增加 token、调用次数和复杂度。

解决方案：为每个里程碑建立功能、性能、工程质量、可演进性四类指标。真实模型测试记录 p50/p95/p99、首 token、错误率、重试和 token；Mock 测试隔离框架开销；再通过“更换供应商、增加日志、增加一种解析风格”等变更实验统计修改文件和业务侵入范围，用真实数据决定是否继续复杂化。

🔧 框架选型与对照实验

- 使用同一道错题讲解与工具任务分别实现 Spring AI 和 LangChain4j 版本，对比 ChatClient/Advisor 与 AI Services/Tool/Content Retriever 的抽象方式、代码量、调试、观测和模型切换成本。
- Spring AI 作为主链路：与 Spring Boot 配置、Actuator、Micrometer、Redis、MCP 集成一致；LangChain4j 保留为独立实验模块，不在主应用重复维护两套生产链路。
- LlamaIndex 不参与 Java 在线问答编排，只发挥 Python 生态在 PDF/OCR、文档节点、分块和增量索引方面的优势，避免三个框架职责重叠。

🎯 项目成果

- 在保留旧业务、前端和数据资产的前提下完成 AI 层演进，形成模型调用、会话记忆、RAG、工具执行和 MCP 运维的清晰边界。
- 柏拉图解析由一次性答案升级为可持久化、可恢复、按学生回答递进的多轮教学；组卷与学习计划从“假 Agent 日志”升级为真实 Tool 调用。
- 沉淀 Spring AI vs LangChain4j 选型报告、架构改造前后对比基准、复用审计、阶段成本收益复盘和可演示的端到端案例，使每项技术都对应一个项目真实问题。
- 建立公开仓库安全基线：旧仓只读隔离、构建产物与密钥不入库、密钥轮换与环境变量注入、MCP 工具脱敏、破坏性操作人审。

🔧 项目经历（续）

StoryCraft：AI 文字冒险游戏（已上线）

Python / FastAPI Next.js / TypeScript Agent Runtime / SSE github.com/baodashuaige/StoryCraft

项目简介：构建可在线游玩的 AI 文字冒险游戏。系统需要让 NPC 对话自然、具有人格和记忆，同时保证概率模型不能绕过规则修改世界状态。项目已提供 Web 与 CLI 入口并部署上线，NPC 对话能力抽取为 npm 包。

核心职责：

- 设计游戏服务端和 Agent Runtime，维护场景、角色、线索、动作、信任值和结局状态；定义 LLM、运行时、工具和前端之间的职责。
- 负责 Prompt、NPC 记忆、动作候选、输出门控、REST API、SSE 流式、前端交互、密钥安全、自动化测试和部署。
- 将 NPC 对话模块抽取为开源 npm 包 @wutiankai/npc-dialogue，提供独立接口与使用示例。

技术难点与解决方案：

难点 1：LLM 叙事自由度与游戏规则确定性冲突

模型擅长生成自然语言，但可能虚构物品、跳过前置条件、替玩家做选择或直接改变角色状态，造成剧情与存档不一致。

解决方案：划分“LLM 生成叙事和候选动作、Agent Runtime 校验动作、确定性代码提交状态”的边界；Prompt 只提供允许感知的信息和可用动作，输出先经过门控校验，再由运行时更新状态，模型不拥有直接写权限。

难点 2：NPC 需要记住玩家行为，但完整历史会持续膨胀并污染角色设定

解决方案：将人格、秘密、信任值、线索、剧情记忆分层组织，稳定设定与动态事件分开存储；每轮只组装当前场景相关信息，避免把全部历史无差别塞入 Prompt，不同角色仅获取其应知信息。

难点 3：流式生成中途失败会导致前端文本与服务端状态不一致

解决方案：SSE 只流式展示候选叙事，状态变更在完整响应校验通过后提交；为连接中断、非法输出和异常动作建立失败路径与测试，保证失败时不产生半完成状态。

项目成果：

- 产品部署至 game.mastertooling.shop，支持公网访问和完整游戏流程。
- 建立状态、动作、对话和接口自动化测试，测试结果 176 passed。
- 形成可复用 NPC 对话 npm 包，并通过实际游戏验证“模型决策与确定性执行分离”的 Agent 设计。

自然语言处理课程项目：中文 NER 与文本生成 课程项目

BERT / LSTM BART CLUENER2020

项目简介：完成中文命名实体识别和文本生成两类任务，覆盖数据清洗、标签处理、模型训练、参数调整、推理与结果评估，用于补足 AI 应用工程之外的模型训练基础。

核心工作：

- 基于 CLUENER2020 完成中文实体标签处理，分别使用 LSTM 与 BERT 训练 NER，对比局部序列建模与预训练上下文表征的差异。
- 基于 BART 进行文本生成微调，理解编码器-解码器、Teacher Forcing、损失计算和生成阶段 beam search / 长度参数的作用。
- 将训练模型、Prompt、RAG、Memory 与 Tool Calling 放在同一能力地图中理解：训练改变模型参数，后四者分别改变输入、外部知识、跨轮状态与真实世界能力。

 开源与工程实践

- @wutiankai/npc-dialogue：从 StoryCraft 中抽取 NPC 对话能力，沉淀可复用接口、角色配置与示例。
- 工程知识沉淀：维护 Agent 原语、Spring AI、Memory、RAG、Tool Calling、MCP 的概念笔记；技术学习始终映射到项目问题和面试表达。
- 增量迁移方法：对旧项目代码做复用审计，保留成熟业务和 API，只重写安全风险、错误语义或阻碍扩展的技术边界。
- 阶段化评测：每个里程碑同步维护计划、对话记录、前后对比、实测数据、成本收益和下一阶段决策，避免只展示最终代码而无法解释取舍。

 荣誉与证书

东华大学软件工程实训二等奖（408Master 项目队长）	
人工智能训练师职业技能培训	预计 2026.08 取证

 教育经历

东华大学（211）	2023.09 – 2027.06
软件工程 本科 · 2027 届 计算机科学与技术学院	上海
主修课程：数据结构、算法、操作系统、计算机网络、数据库、软件工程、自然语言处理。	
学习方向：AI 应用架构、Java 后端、检索增强生成、Agent 工具与记忆、模型训练基础。	

 目标岗位匹配

- AI 应用工程师：能够完成模型接入、Prompt、RAG、Memory、Tool、MCP、评测观测与产品交付，不把 API 调用等同于 Agent。
- Java 后端开发：具备 Spring Boot、MyBatis、MySQL、Redis、接口兼容、数据迁移、安全和可观测实践，能够将 AI 能力放入可维护的业务系统。
- Agent 工程方向：有两个不同场景的状态与工具边界实践：教育平台侧重受监督工具和长期学习状态，游戏侧重叙事自由度与确定性世界状态。